

Relatório Projeto Integrador II

Curso: Ciência da Computação

Semestre: 6º

Título do Projeto: Ingenia — Plataforma Digital de Ensino Introdutório de Programação

Integrantes: Gabriel Marques, Thales Rassi, Matheus Moraes e Henrique Lessa

Instituição: CEUB

Introdução

Este projeto foi desenvolvido no contexto da disciplina **Projeto Integrador II**, com o objetivo de propor e implementar uma solução tecnológica para o ensino introdutório de programação. A organização atendida é o contexto educacional do ensino fundamental — **escolas públicas e privadas e estudantes em uso individual em casa** —, que carece de ferramentas acessíveis para iniciar adolescentes na programação.

A **Ingenia** é uma plataforma web que transforma o "começar a programar" em uma jornada clara e guiada, voltada a estudantes do **8º e 9º ano**. Nela, o aluno assiste a videoaulas, lê o conteúdo escrito, resolve exercícios em Python diretamente no navegador, recebe feedback automático imediato e acompanha seu progresso ao longo de uma trilha de módulos. A plataforma também atende professores, com acompanhamento de turmas e desempenho, e administradores, com gestão de conteúdo e usuários.

Objetivo geral do sistema: oferecer uma plataforma estruturada e intuitiva para o ensino introdutório de programação, com foco em aprendizagem ativa por meio de conteúdo guiado, prática e feedback imediato.

Problema Identificado

Descrição do problema. Oferecer uma introdução prática, guiada e acessível à programação para estudantes sem experiência prévia é difícil: não existe um ambiente único que combine teoria, prática e retorno imediato sobre o aprendizado, adequado à faixa etária do ensino fundamental.

Situação atual. O ensino introdutório de programação costuma depender de ferramentas fragmentadas — vídeos em uma plataforma, editores de código instalados localmente (com instalação e configuração complexas para iniciantes) e correção manual de exercícios pelo professor. Isso gera atrito, atrasa o feedback e dificulta o acompanhamento do progresso de cada aluno.

Impacto do problema. A barreira técnica e a ausência de feedback imediato desmotivam alunos iniciantes e sobrecarregam o professor com correções manuais. O resultado é evasão, baixa

autonomia do estudante e pouca visibilidade do desempenho da turma.

Justificativa

Por que o projeto é importante. A programação é uma competência cada vez mais central, e introduzi-la cedo, de forma acessível, amplia o repertório e a autonomia dos estudantes. Concentrar conteúdo, prática e correção automática em um único ambiente web — sem instalação — remove a principal barreira de entrada para iniciantes.

Impacto esperado. Reduzir o tempo entre escrever código e entender se ele está correto (feedback imediato), aumentar o engajamento por meio de uma trilha visual de progresso e liberar o professor das correções manuais, permitindo foco na intervenção pedagógica.

Benefícios para a organização (contexto escolar). Maior organização do conteúdo pedagógico, padronização da correção, acompanhamento objetivo do desempenho por turma e por aluno, e possibilidade de uso tanto em sala quanto em casa.

Público-Alvo

O sistema possui três perfis de usuário:

- **Aluno** — estudante do 8º e 9º ano, iniciante em programação. Percorre a trilha, consome aulas, resolve exercícios e acompanha o próprio progresso.
 - **Professor** — docente do ensino fundamental. Organiza turmas, matricula alunos e acompanha o desempenho coletivo e individual.
 - **Administrador** — responsável pela plataforma. Gerencia módulos, aulas, exercícios e usuários, e visualiza métricas gerais.
-

Objetivos do Sistema

Objetivo Geral

Desenvolver uma plataforma web para ensino introdutório de programação, com trilha de módulos, editor de código no navegador, correção automática e acompanhamento de progresso por alunos, professores e administradores.

Objetivos Específicos

- Organizar o ensino em módulos progressivos (variáveis, condicionais, loops, funções, etc.).
- Permitir assistir videoaulas, ler material escrito e resolver exercícios em um único ambiente.
- Disponibilizar correção automática que executa testes internos e informa se a solução atende ao esperado.

- Fornecer feedback pedagógico imediato sem expor a resposta completa.
 - Oferecer ao professor um painel de acompanhamento de progresso e desempenho por turma.
 - Disponibilizar ao administrador o controle completo sobre conteúdo e usuários.
 - Garantir autenticação, controle de permissões por perfil e proteção básica de dados.
-

Backlog do Produto

O backlog foi construído de forma coerente com os requisitos definidos, priorizado e distribuído em sprints.

ID	Tipo	User Story / Feature	Prioridade	Sprint
US01	Feature	Modelar e implementar a estrutura inicial de dados (usuários, currículo, turmas, submissões, progresso) + Django Admin e seed	Alta	1
US02	User Story	Como usuário, quero fazer login seguro e ser direcionado à minha área conforme meu perfil	Alta	2
US03	User Story	Como aluno, quero me cadastrar na plataforma e recuperar minha senha por e-mail	Alta	2
US04	Feature	Implementar controle de permissões por perfil (aluno/professor/admin) e guards de rota	Alta	2
US05	User Story	Como administrador, quero criar e organizar módulos, aulas e videoaulas	Alta	3
US06	User Story	Como administrador, quero cadastrar exercícios com casos de teste e publicar o conteúdo	Alta	3
US07	User Story	Como administrador, quero gerenciar usuários por perfil	Média	3
US08	User Story	Como aluno, quero percorrer a trilha de módulos e acessar aulas com vídeo e conteúdo	Alta	4
US09	User Story	Como aluno, quero resolver exercícios em Python no navegador e receber correção automática	Alta	4
US10	Feature	Implementar o motor de correção client-side (Skulpt) com avaliação por casos de teste e feedback	Alta	4
US11	User Story	Como aluno, quero acompanhar meu progresso e o histórico das minhas submissões	Média	5
US12	User Story	Como professor, quero criar turmas e matricular/remover alunos	Alta	5
US13	User Story	Como professor, quero acompanhar o progresso coletivo e individual da turma	Alta	6
US14	Feature	Implementar medidas de segurança (rate limiting, validações, CORS, auditoria)	Média	6
US15	Feature	Validar as jornadas via testes E2E e ajustar UX/responsividade	Média	7
US16	Feature	Realizar o deploy em produção com Docker + Traefik (TLS)	Alta	7

Cronograma de Entrega

Cronograma planejado por sprint (cada "X" indica a sprint em que a atividade foi desenvolvida).

Entrega / Atividade	S1	S2	S3	S4	S5	S6	S7	S8
Levantamento de requisitos	X							
Definição e priorização do backlog	X	X						
Modelagem do banco de dados	X							
Configuração do ambiente (repo, Docker, deploy base)	X	X						
Autenticação, perfis e permissões		X						
Administração de conteúdo (CRUD de currículo e usuários)			X					
Experiência do aluno + correção automática (Skulpt)				X				
Progresso e histórico de submissões					X			
Experiência do professor (turmas e acompanhamento)					X	X		
Segurança, validações e ajustes						X		
Testes E2E e refinamento de UX						X	X	
Deploy estável e documentação final							X	X

Requisitos do Sistema

Requisitos Funcionais

- **RF01** — O sistema deve permitir o cadastro de alunos e o login seguro de todos os perfis.
- **RF02** — O sistema deve permitir a recuperação e redefinição de senha por e-mail.
- **RF03** — O sistema deve redirecionar o usuário para a área correspondente ao seu perfil após o login.
- **RF04** — O administrador deve poder criar, editar, publicar e remover módulos, aulas e videoaulas.
- **RF05** — O administrador deve poder cadastrar exercícios e seus casos de teste (visíveis e ocultos).
- **RF06** — O administrador deve poder gerenciar usuários por perfil.
- **RF07** — O aluno deve poder navegar pela trilha de módulos e acessar aulas com vídeo e conteúdo escrito.
- **RF08** — O aluno deve poder escrever e executar código Python diretamente no navegador.
- **RF09** — O sistema deve corrigir automaticamente o código submetido contra os casos de teste e exibir resultado e feedback.
- **RF10** — O sistema deve registrar as submissões e calcular o progresso (aula, exercício, módulo) automaticamente.
- **RF11** — O aluno deve poder consultar seu progresso e o histórico de submissões.

- **RF12** — O professor deve poder criar/editar turmas e matricular/remover alunos.
- **RF13** — O professor deve poder visualizar o progresso coletivo e individual dos alunos de suas turmas.
- **RF14** — O administrador deve visualizar métricas agregadas em um dashboard.

Requisitos Não Funcionais

- **RNF01** — O sistema deve ser acessível via navegador web, com foco em uso desktop.
 - **RNF02** — A autenticação deve usar JWT (token de acesso + refresh), com bloqueio de contas inativas.
 - **RNF03** — O controle de acesso deve ser baseado em perfis (RBAC), no backend e no frontend.
 - **RNF04** — O sistema deve garantir isolamento de dados (aluno vê apenas seus dados; professor, apenas suas turmas).
 - **RNF05** — A execução do código do aluno deve ocorrer de forma isolada no navegador, com limite de tempo, sem comprometer o servidor.
 - **RNF06** — O feedback da correção deve ser praticamente imediato.
 - **RNF07** — A comunicação cliente-servidor deve ocorrer via API REST sobre HTTPS (TLS).
 - **RNF08** — Os dados devem ser persistidos em PostgreSQL, com integridade referencial.
 - **RNF09** — O sistema deve possuir cobertura de testes (unitários, de integração e E2E).
 - **RNF10** — A interface deve ser simples e guiada, adequada a alunos iniciantes.
-

Modelagem do Banco de Dados

O modelo conceitual organiza as entidades em cinco domínios: **contas/usuários, currículo, turmas, submissões e progresso**.

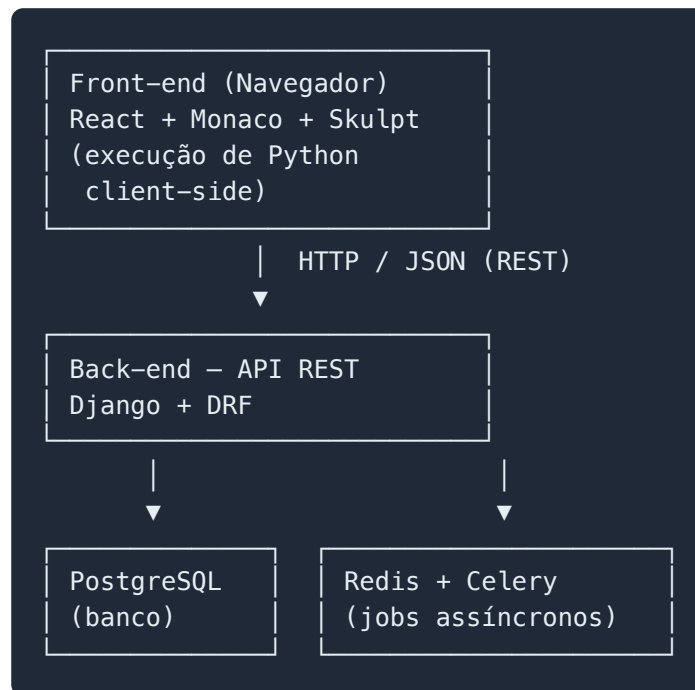
Principais entidades

- **User** — usuário base da plataforma (e-mail único, papel/role e status da conta). Possui um perfil especializado conforme o papel: **StudentProfile**, **TeacherProfile** ou **AdminProfile**.
- **Module** — módulo temático da trilha (título, ordem e status de publicação).
- **Lesson** — aula dentro de um módulo (conteúdo escrito, ordem). Possui uma **VideoLesson** associada (URL e duração do vídeo).
- **Exercise** — exercício vinculado a uma aula (enunciado, mensagem de apoio).
- **ExerciseTestCase** — caso de teste de um exercício (entrada, saída esperada, oculto ou visível).
- **ClassGroup** — turma de um professor; **ClassEnrollment** — matrícula que associa um aluno a uma turma.
- **Submission** — submissão de código de um aluno para um exercício; **SubmissionResult** — resultado consolidado (acertos, erros, feedback).

- **StudentModuleProgress / StudentLessonProgress / StudentExerciseProgress** — progresso do aluno por módulo, aula e exercício.
-

Arquitetura do Sistema

O sistema é um **monorepo full-stack** com front-end e back-end desacoplados, comunicando-se via **API REST**.



- **Front-end** — desenvolvido em **React + TypeScript** (Vite), com a biblioteca de componentes **Mantine** e **TanStack Query** para o estado de servidor. O editor de código é o **Monaco** e a execução de Python ocorre no próprio navegador via **Skulpt**.
- **Back-end** — **Django + Django REST Framework** (Python), organizado em camadas (Models → Selectors → Services/UseCases → Serializers → Views), com autenticação **JWT**.
- **Banco de dados** — **PostgreSQL**. **Redis + Celery** dão suporte a tarefas assíncronas.
- **Infraestrutura** — orquestração com **Docker Compose** e, em produção, **Traefik** com TLS automático (Let's Encrypt).

Decisão técnica de destaque: a correção automática é executada **no navegador** (Skulpt), e não em um servidor de execução. Isso elimina a complexidade e o risco de segurança de um *sandbox* server-side e torna o feedback praticamente instantâneo — o back-end apenas persiste o resultado já avaliado.

Funcionalidades Desenvolvidas

A seguir, as principais funcionalidades implementadas. *(As capturas de tela de cada funcionalidade serão inseridas neste tópico.)*

Autenticação e Conta

- **Login por perfil** — autenticação com e-mail/senha e redirecionamento automático para a área do aluno, professor ou administrador.
- **Cadastro e recuperação de senha** — registro público de alunos e fluxo de redefinição de senha por e-mail.

Área do Aluno

- **Trilha de aprendizagem** — dashboard com módulos, progresso visual e atalho "Continuar de onde parei".
- **Aula** — player de vídeo (YouTube/Vimeo), conteúdo em Markdown e lista de exercícios.
- **Exercício com correção automática** — editor Monaco, execução de Python no navegador (Skulpt), avaliação contra casos de teste e feedback pedagógico imediato.
- **Progresso e histórico** — visão consolidada do avanço e histórico de submissões com filtros.

Área do Professor

- **Gestão de turmas** — criação/edição de turmas e matrícula de alunos por busca.
- **Acompanhamento de desempenho** — progresso coletivo da turma e progresso individual por módulo, aula e exercício.

Área do Administrador

- **Gestão de conteúdo** — CRUD completo de módulos, aulas, videoaulas, exercícios e casos de teste, com fluxo de publicação.
- **Gestão de usuários** — criação e edição de usuários por perfil.
- **Dashboard** — métricas agregadas da plataforma.

Demonstração do Sistema

Link de Acesso

 <https://ingenia.synaptha.com>

Logins de teste

Perfil	Usuário	Senha
Administrador	admin@hub.dev	admin123
Professor	teacher1@hub.dev	teacher123
Aluno	user@hub.dev	user123

Resultados e Impactos

Benefícios observados. O MVP foi entregue completo e está em funcionamento: uma trilha com módulos progressivos, correção automática de exercícios com feedback imediato e três perfis integrados (aluno, professor e administrador). O conteúdo é totalmente gerenciável pelo administrador, sem necessidade de alterar código.

Mudanças geradas. A plataforma reúne, em um único ambiente web sem instalação, o que antes estava fragmentado: teoria (vídeo + texto), prática (editor no navegador) e correção (automática). Isso reduz a barreira de entrada do aluno iniciante e dá ao professor visibilidade objetiva do desempenho da turma, eliminando a correção manual.

Possibilidades futuras. Evoluir itens fora do escopo do MVP conforme demanda — gamificação, responsividade para dispositivos móveis, certificação de conclusão e suporte a outras linguagens de programação —, além de aprofundar o *hardening* de segurança e a infraestrutura de produção.

Considerações Finais

Síntese do projeto. A Ingenia cumpriu seu objetivo de oferecer um ambiente único, guiado e acessível para o ensino introdutório de programação, cobrindo as jornadas de aluno, professor e administrador, com correção automática e acompanhamento de progresso.

Avaliação do resultado. Todas as funcionalidades essenciais (MVP) planejadas foram implementadas e validadas, e a aplicação está disponível em produção. As decisões de arquitetura — camadas no back-end, domínios espelhados no front-end e execução de código no cliente — mostraram-se acertadas em termos de clareza, testabilidade e segurança.

Contribuição acadêmica. O projeto consolidou aprendizados em engenharia de software full-stack: modelagem de dados, arquitetura em camadas, API REST, autenticação e autorização, integração front-end/back-end, execução de código em ambiente controlado, testes automatizados (unitários e E2E) e implantação com contêineres. Também exercitou práticas ágeis de planejamento, com backlog priorizado e entregas incrementais por sprint.

Referências

- Django — Documentação oficial. <https://docs.djangoproject.com/>
- Django REST Framework. <https://www.django-rest-framework.org/>
- Simple JWT (autenticação JWT para DRF). <https://django-rest-framework-simplejwt.readthedocs.io/>
- React. <https://react.dev/>
- Vite. <https://vitejs.dev/>
- Mantine (biblioteca de componentes React). <https://mantine.dev/>

- TanStack Query. <https://tanstack.com/query/>
- Skulpt (interpretador Python em JavaScript). <https://skulpt.org/>
- Monaco Editor. <https://microsoft.github.io/monaco-editor/>
- PostgreSQL. <https://www.postgresql.org/docs/>
- Docker. <https://docs.docker.com/>
- Playwright. <https://playwright.dev/>